DSA Questions + Patterns

500 Curated Problems for FAANG & Top-Tier Interview Preparation

Covering Arrays, Strings, Trees, Graphs, DP, and 20+ Core Topics

Introduction

This document is a comprehensive, formally curated collection of 500 Data Structures and Algorithms (DSA) questions designed for candidates preparing for technical interviews at top technology companies including Google, Amazon, Apple, Netflix, Meta (FAANG), Microsoft, and other tier-1 organisations. Each question is accompanied by its associated algorithmic pattern, enabling a pattern-first study approach that dramatically improves problem-solving speed and accuracy during interviews.

Questions are organized by topic and sub-topic. Difficulty is annotated as [Easy], [Medium], or [Hard]. Master the patterns, not just the problems.

Section 1: Arrays (Q1 – Q50)

1.1 Two Pointers

Pattern: Two Pointers — use two indices moving toward/away from each other to reduce O(n²) to O(n).

1. Two Sum (sorted array) **[Easy]**
2. Three Sum **[Medium]**
3. Four Sum **[Medium]**
4. Container With Most Water **[Medium]**
5. Trapping Rain Water **[Hard]**
6. Remove Duplicates from Sorted Array **[Easy]**
7. Move Zeroes to End **[Easy]**
8. Sort Colors (Dutch National Flag) **[Medium]**
9. Minimum Size Subarray Sum **[Medium]**
10. Squares of a Sorted Array **[Easy]**

1.2 Sliding Window

Pattern: Sliding Window — maintain a window of elements; expand/shrink to satisfy a constraint.

11. Longest Substring Without Repeating Characters **[Medium]**
12. Maximum Sum Subarray of Size K **[Easy]**
13. Fruit Into Baskets (at most 2 distinct) **[Medium]**
14. Minimum Window Substring **[Hard]**
15. Longest Subarray with Ones after Replacement **[Medium]**
16. Permutation in String **[Medium]**
17. Find All Anagrams in a String **[Medium]**
18. Substring with Concatenation of All Words **[Hard]**
19. Max Consecutive Ones III **[Medium]**
20. Number of Subarrays with Product Less Than K **[Medium]**

1.3 Prefix Sum / Difference Array

Pattern: Prefix Sum — precompute cumulative sums for O(1) range queries.

21. Subarray Sum Equals K **[Medium]**
22. Range Sum Query — Immutable **[Easy]**
23. Continuous Subarray Sum (multiple of k) **[Medium]**
24. Product of Array Except Self **[Medium]**
25. Find Pivot Index **[Easy]**
26. Count Subarrays with Equal 0s and 1s **[Medium]**
27. Subarray Sums Divisible by K **[Medium]**
28. Minimum Operations to Reduce X to Zero **[Medium]**
29. Running Sum of 1D Array **[Easy]**
30. Random Pick with Weight **[Medium]**

1.4 Kadane's / Greedy on Arrays

Pattern: Kadane's Algorithm — track local and global maximum for maximum subarray problems.

31. Maximum Subarray (Kadane's) **[Medium]**
32. Maximum Product Subarray **[Medium]**
33. Best Time to Buy and Sell Stock **[Easy]**
34. Best Time to Buy and Sell Stock II **[Medium]**
35. Jump Game **[Medium]**
36. Jump Game II (minimum jumps) **[Medium]**
37. Gas Station **[Medium]**
38. Candy Distribution **[Hard]**
39. Partition Labels **[Medium]**
40. Minimum Number of Arrows to Burst Balloons **[Medium]**

1.5 Sorting & Searching in Arrays

Pattern: Sorting + Binary Search — sort once, then binary-search for O(n log n) solutions.

41. Merge Intervals **[Medium]**
42. Insert Interval **[Medium]**
43. Meeting Rooms II (min conference rooms) **[Medium]**
44. Non-overlapping Intervals **[Medium]**
45. Largest Number (custom sort) **[Medium]**
46. Kth Largest Element in an Array **[Medium]**
47. Find Minimum in Rotated Sorted Array **[Medium]**
48. Search in Rotated Sorted Array **[Medium]**
49. Count of Smaller Numbers After Self **[Hard]**
50. Find the Duplicate Number **[Medium]**

Section 2: Strings (Q51 – Q90)

2.1 String Manipulation & Hashing

Pattern: HashMap / Frequency Count — count character frequencies to compare or find patterns.

51. Valid Anagram **[Easy]**
52. Group Anagrams **[Medium]**
53. Longest Common Prefix **[Easy]**
54. Reverse Words in a String **[Medium]**
55. String to Integer (atoi) **[Medium]**
56. Roman to Integer **[Easy]**
57. Integer to Roman **[Medium]**
58. Count and Say **[Medium]**
59. ZigZag Conversion **[Medium]**
60. Multiply Strings **[Medium]**
61. Longest Palindrome (by rearrangement) **[Easy]**
62. First Unique Character in a String **[Easy]**

2.2 Sliding Window on Strings

Pattern: Variable-size Sliding Window — track character counts inside a window.

63. Longest Repeating Character Replacement **[Medium]**
64. Minimum Window Substring (revisited with string focus) **[Hard]**
65. Longest Substring with At Most K Distinct Characters **[Medium]**
66. Longest Substring with At Most Two Distinct Characters **[Medium]**
67. Longest Palindromic Substring **[Medium]**
68. Palindromic Substrings (count all) **[Medium]**

2.3 Pattern Matching

Pattern: KMP / Rabin-Karp / Z-Algorithm — efficient O(n+m) substring search.

69. Implement strStr() / Needle in Haystack **[Easy]**
70. Repeated Substring Pattern **[Easy]**
71. Shortest Palindrome (KMP) **[Hard]**
72. Longest Happy Prefix (KMP failure function) **[Hard]**
73. Find the Index of the First Occurrence in a String **[Easy]**
74. Wildcard Matching **[Hard]**
75. Regular Expression Matching **[Hard]**
76. Word Break **[Medium]**
77. Word Break II **[Hard]**
78. Decode Ways **[Medium]**

2.4 Stack-based String Problems

Pattern: Monotonic Stack / Explicit Stack — process characters left-to-right with a stack.

79. Valid Parentheses **[Easy]**
80. Minimum Remove to Make Valid Parentheses **[Medium]**
81. Score of Parentheses **[Medium]**
82. Decode String (k[encoded_string]) **[Medium]**
83. Remove All Adjacent Duplicates in String **[Easy]**
84. Remove K Digits **[Medium]**
85. Basic Calculator II **[Medium]**
86. Evaluate Reverse Polish Notation **[Medium]**
87. Remove Duplicate Letters (lexicographically smallest) **[Medium]**
88. Largest Rectangle in Histogram (string of bars) **[Hard]**

Section 3: Linked Lists (Q91 – Q120)

3.1 Fast & Slow Pointers

Pattern: Floyd's Cycle Detection — two pointers at different speeds to detect cycles and midpoints.

89. Linked List Cycle Detection **[Easy]**
90. Linked List Cycle II (entry point) **[Medium]**
91. Find the Middle of Linked List **[Easy]**
92. Happy Number (cycle in sequence) **[Easy]**
93. Palindrome Linked List **[Easy]**
94. Reorder List (L0→Ln→L1→Ln-1) **[Medium]**

3.2 Reversal & Merging

Pattern: In-place Reversal — reverse sublists or entire lists using pointer manipulation.

95. Reverse a Linked List **[Easy]**
96. Reverse Linked List II (sub-list) **[Medium]**
97. Reverse Nodes in k-Group **[Hard]**
98. Rotate List by k **[Medium]**
99. Merge Two Sorted Lists **[Easy]**
100. Merge K Sorted Lists **[Hard]**
101. Sort List (merge sort on LL) **[Medium]**
102. Partition List around value x **[Medium]**
103. Remove Nth Node from End **[Medium]**
104. Delete Node in a Linked List (no head) **[Easy]**
105. Odd Even Linked List **[Medium]**
106. Intersection of Two Linked Lists **[Easy]**
107. Flatten a Multilevel Doubly Linked List **[Medium]**
108. Copy List with Random Pointer **[Medium]**
109. LRU Cache (design problem) **[Medium]**

3.3 Advanced Linked List

110. LFU Cache **[Hard]**
111. Design Skiplist **[Hard]**
112. Add Two Numbers (LL representation) **[Medium]**
113. Add Two Numbers II (no reversal) **[Medium]**
114. Swap Nodes in Pairs **[Medium]**
115. Remove Duplicates from Sorted List II **[Medium]**
116. Next Greater Node in Linked List **[Medium]**
117. Insert into a Sorted Circular Linked List **[Medium]**
118. Convert Binary Number in LL to Integer **[Easy]**

Section 4: Stacks & Queues (Q121 – Q145)

4.1 Monotonic Stack

Pattern: Monotonic Stack — maintain a stack in increasing/decreasing order for next-greater/smaller queries.

119. Next Greater Element I **[Easy]**
120. Next Greater Element II (circular) **[Medium]**
121. Daily Temperatures **[Medium]**
122. Largest Rectangle in Histogram **[Hard]**
123. Maximal Rectangle in Binary Matrix **[Hard]**
124. Sum of Subarray Minimums **[Medium]**
125. Maximum Width Ramp **[Medium]**
126. Online Stock Span **[Medium]**
127. 132 Pattern **[Medium]**
128. Asteroid Collision **[Medium]**

4.2 Queue / Deque Patterns

Pattern: Monotonic Deque — maintain a deque for sliding window max/min in O(n).

129. Sliding Window Maximum **[Hard]**
130. First Negative in Every Window of Size K **[Medium]**
131. Implement Stack using Queues **[Easy]**
132. Implement Queue using Stacks **[Easy]**
133. Design Circular Queue **[Medium]**
134. Design Circular Deque **[Medium]**
135. Max of Min for Every Window Size **[Hard]**
136. Shortest Subarray with Sum at Least K **[Hard]**
137. Jump Game VI (DP + deque) **[Medium]**
138. Constrained Subsequence Sum **[Hard]**
139. Task Scheduler **[Medium]**
140. Number of Recent Calls **[Easy]**
141. Dota2 Senate **[Medium]**
142. Reveal Cards in Increasing Order **[Medium]**
143. Rotten Oranges (BFS) **[Medium]**

Section 5: Binary Search (Q146 – Q175)

5.1 Classic Binary Search

Pattern: Binary Search — repeatedly halve the search space on a monotonic function.

144. Binary Search (basic) **[Easy]**
145. First Bad Version **[Easy]**
146. Search Insert Position **[Easy]**
147. Count of Range Sum (advanced) **[Hard]**
148. Find Peak Element **[Medium]**
149. Find Minimum in Rotated Sorted Array II (duplicates) **[Hard]**
150. Search a 2D Matrix **[Medium]**
151. Search a 2D Matrix II **[Medium]**
152. Kth Smallest Element in Sorted Matrix **[Medium]**
153. Find K Closest Elements **[Medium]**

5.2 Binary Search on Answer

Pattern: Parametric Search — binary search on the answer space, validate with a feasibility function.

154. Capacity to Ship Packages Within D Days **[Medium]**
155. Split Array Largest Sum **[Hard]**
156. Koko Eating Bananas **[Medium]**
157. Minimum Number of Days to Make m Bouquets **[Medium]**
158. Magnetic Force Between Two Balls **[Medium]**
159. Find the Smallest Divisor Given a Threshold **[Medium]**
160. Allocate Minimum Number of Pages **[Medium]**
161. Aggressive Cows (SPOJ classic) **[Medium]**
162. Painters Partition Problem **[Hard]**
163. EKO — Cutting Trees (binary search on height) **[Medium]**
164. Median of Two Sorted Arrays **[Hard]**
165. K-th Smallest Prime Fraction **[Hard]**
166. Find K-th Smallest Pair Distance **[Hard]**
167. Count of Pairs with Sum Less Than Target **[Easy]**
168. Maximum Running Time of N Computers **[Hard]**

5.3 Bisect Tricks

169. Longest Increasing Subsequence (O(n log n)) **[Medium]**
170. Russian Doll Envelopes **[Hard]**
171. Count of Smaller Numbers After Self (BIT/BST) **[Hard]**
172. Minimum Operations to Make Array Increasing **[Easy]**
173. Maximum Profit in Job Scheduling **[Hard]**
174. Number of Longest Increasing Subsequences **[Medium]**

Section 6: Trees (Q176 – Q230)

6.1 Tree Traversals

Pattern: DFS (Pre/In/Post-order) & BFS (Level-order) — foundation of nearly all tree problems.

175. Binary Tree Inorder Traversal (iterative) **[Easy]**
176. Binary Tree Preorder Traversal (iterative) **[Easy]**
177. Binary Tree Postorder Traversal (iterative) **[Easy]**
178. Binary Tree Level Order Traversal **[Medium]**
179. Binary Tree Zigzag Level Order **[Medium]**
180. Binary Tree Right Side View **[Medium]**
181. Average of Levels in Binary Tree **[Easy]**
182. N-ary Tree Level Order Traversal **[Medium]**
183. Vertical Order Traversal **[Hard]**
184. Boundary Traversal of Binary Tree **[Medium]**

6.2 Binary Tree Properties

Pattern: Tree Recursion — solve sub-problems on left/right subtrees and combine results.

185. Maximum Depth of Binary Tree **[Easy]**
186. Minimum Depth of Binary Tree **[Easy]**
187. Diameter of Binary Tree **[Easy]**
188. Balanced Binary Tree **[Easy]**
189. Same Tree **[Easy]**
190. Symmetric Tree **[Easy]**
191. Invert Binary Tree **[Easy]**
192. Count Complete Tree Nodes **[Medium]**
193. Sum of Left Leaves **[Easy]**
194. Path Sum **[Easy]**
195. Path Sum II (all paths) **[Medium]**
196. Binary Tree Maximum Path Sum **[Hard]**
197. Sum Root to Leaf Numbers **[Medium]**
198. Flatten Binary Tree to Linked List **[Medium]**
199. Populating Next Right Pointers **[Medium]**

6.3 Binary Search Trees (BST)

Pattern: BST Property — left < root < right; enables O(log n) search, insert, delete.

200. Validate Binary Search Tree **[Medium]**
201. Kth Smallest Element in a BST **[Medium]**
202. Lowest Common Ancestor of BST **[Easy]**
203. Lowest Common Ancestor of Binary Tree **[Medium]**
204. Convert Sorted Array to BST **[Easy]**
205. Convert BST to Greater Tree **[Medium]**
206. Insert into a BST **[Medium]**
207. Delete Node in a BST **[Medium]**
208. Recover Binary Search Tree (two nodes swapped) **[Hard]**
209. Unique Binary Search Trees (count) **[Medium]**
210. Unique Binary Search Trees II (generate all) **[Medium]**
211. Serialize and Deserialize Binary Tree **[Hard]**
212. Binary Tree Cameras **[Hard]**
213. Construct BST from Preorder Traversal **[Medium]**
214. Two Sum IV in BST **[Easy]**

6.4 Segment Trees & BITs (Advanced)

Pattern: Segment Tree / Fenwick Tree — O(log n) range queries and point updates.

215. Range Sum Query — Mutable (Segment Tree / BIT) **[Medium]**
216. Range Minimum Query **[Medium]**
217. Count of Range Sum (Merge Sort / BIT) **[Hard]**
218. Number of Longest Increasing Subsequences (Segment Tree) **[Medium]**
219. The Skyline Problem **[Hard]**
220. My Calendar I, II, III **[Medium]**
221. Rectangle Area II (coordinate compression) **[Hard]**
222. Falling Squares **[Hard]**
223. Interval List Intersections **[Medium]**
224. Data Stream as Disjoint Intervals **[Hard]**

6.5 Tries

Pattern: Trie (Prefix Tree) — efficient prefix search and autocomplete in O(L) per query.

225. Implement Trie (Prefix Tree) **[Medium]**
226. Word Search II (Trie + DFS) **[Hard]**
227. Design Add and Search Words Data Structure **[Medium]**
228. Replace Words with Root (Trie) **[Medium]**
229. Maximum XOR of Two Numbers (Trie) **[Medium]**
230. Palindrome Pairs (Trie) **[Hard]**

Section 7: Heaps & Priority Queues (Q232 – Q255)

7.1 Top-K Pattern

Pattern: Min-Heap of size K — maintain the K largest elements efficiently.

231. Kth Largest Element in a Stream **[Easy]**
232. Top K Frequent Elements **[Medium]**
233. Top K Frequent Words **[Medium]**
234. K Closest Points to Origin **[Medium]**
235. Find K Pairs with Smallest Sums **[Medium]**
236. Kth Smallest Element in Sorted Matrix (Heap) **[Medium]**
237. Sort Characters By Frequency **[Medium]**
238. Reorganize String **[Medium]**
239. Task Scheduler (Heap approach) **[Medium]**
240. Maximum Frequency Stack **[Hard]**

7.2 Two-Heap Pattern

Pattern: Two Heaps — one max-heap and one min-heap to find the median dynamically.

241. Find Median from Data Stream **[Hard]**
242. Sliding Window Median **[Hard]**
243. IPO (Maximize Capital) **[Hard]**
244. Minimum Cost to Connect Sticks **[Medium]**
245. Smallest Range Covering Elements from K Lists **[Hard]**
246. Ugly Number II **[Medium]**
247. Super Ugly Number **[Medium]**
248. Meeting Rooms III (heap-based room assignment) **[Hard]**
249. Employee Free Time **[Hard]**
250. Process Tasks Using Servers **[Medium]**
251. Single-Threaded CPU **[Medium]**
252. Furthest Building You Can Reach **[Medium]**

Section 8: Graphs (Q256 – Q310)

8.1 BFS (Shortest Path / Level-order)

Pattern: BFS — explore nodes level by level; guarantees shortest path in unweighted graphs.

253. Number of Islands **[Medium]**
254. Max Area of Island **[Medium]**
255. Flood Fill **[Easy]**
256. Pacific Atlantic Water Flow **[Medium]**
257. 01 Matrix (multi-source BFS) **[Medium]**
258. Rotting Oranges **[Medium]**
259. Walls and Gates (multi-source BFS) **[Medium]**
260. Shortest Path in Binary Matrix **[Medium]**
261. Snakes and Ladders **[Medium]**
262. Open the Lock **[Medium]**
263. Word Ladder **[Hard]**
264. Word Ladder II (all shortest paths) **[Hard]**
265. Minimum Knight Moves **[Medium]**
266. Bus Routes (BFS on routes) **[Hard]**
267. Cut Off Trees for Golf Event **[Hard]**

8.2 DFS / Backtracking on Graphs

Pattern: DFS with Visited Set — explore all paths; detect cycles and connected components.

268. Clone Graph **[Medium]**
269. Course Schedule (cycle detection) **[Medium]**
270. Course Schedule II (topological sort) **[Medium]**
271. Number of Connected Components in Undirected Graph **[Medium]**
272. Graph Valid Tree **[Medium]**
273. Redundant Connection (Union-Find) **[Medium]**
274. Accounts Merge (Union-Find / DFS) **[Medium]**
275. All Paths from Source to Target **[Medium]**
276. Is Graph Bipartite? **[Medium]**
277. Possible Bipartition **[Medium]**

8.3 Topological Sort

Pattern: Kahn's Algorithm / DFS Post-order — linear ordering of nodes in a DAG.

278. Alien Dictionary **[Hard]**
279. Sequence Reconstruction **[Medium]**
280. Minimum Height Trees **[Medium]**
281. Find Eventual Safe States **[Medium]**
282. Parallel Courses (min semesters) **[Medium]**
283. Sort Items by Groups Respecting Dependencies **[Hard]**

8.4 Union-Find (Disjoint Set Union)

Pattern: DSU with Path Compression + Union by Rank — near-O(1) amortised operations.

284. Find if Path Exists in Graph **[Easy]**
285. Number of Provinces **[Medium]**
286. Redundant Connection II (directed) **[Hard]**
287. Making a Large Island **[Hard]**
288. Swim in Rising Water **[Hard]**
289. Earliest Moment When Everyone Become Friends **[Medium]**
290. Satisfiability of Equality Equations **[Medium]**
291. Remove Max Number of Edges to Keep Graph Fully Traversable **[Hard]**

8.5 Shortest Path Algorithms

Pattern: Dijkstra / Bellman-Ford / Floyd-Warshall — weighted shortest paths.

292. Network Delay Time (Dijkstra) **[Medium]**
293. Path with Maximum Probability **[Medium]**
294. Cheapest Flights Within K Stops (Bellman-Ford) **[Medium]**
295. Find the City with Smallest Number of Neighbors (Floyd-Warshall) **[Medium]**
296. Path with Minimum Effort (Dijkstra / Binary Search) **[Medium]**
297. Number of Ways to Arrive at Destination **[Medium]**
298. Minimum Weighted Subgraph with Required Paths **[Hard]**
299. Minimum Score of a Path Between Two Cities **[Medium]**
300. Reachable Nodes in Subdivided Graph **[Hard]**
301. K-th Shortest Path (Yen's Algorithm concept) **[Hard]**

8.6 Minimum Spanning Tree

Pattern: Kruskal's / Prim's — greedily build a spanning tree with minimum total edge weight.

302. Min Cost to Connect All Points (Prim's) **[Medium]**
303. Optimize Water Distribution in a Village **[Hard]**
304. Critical Connections in a Network (Bridges) **[Hard]**
305. Minimum Cost to Reach City with Tolls **[Medium]**
306. Find Critical and Pseudo-Critical Edges in MST **[Hard]**

Section 9: Dynamic Programming (Q311 – Q390)

9.1 1-D DP (Linear Recurrences)

Pattern: 1-D DP — state is a single index; transition from previous states.

307. Climbing Stairs **[Easy]**
308. House Robber **[Medium]**
309. House Robber II (circular) **[Medium]**
310. House Robber III (tree DP) **[Medium]**
311. Min Cost Climbing Stairs **[Easy]**
312. Fibonacci Number **[Easy]**
313. Tribonacci Number **[Easy]**
314. Coin Change (minimum coins) **[Medium]**
315. Coin Change II (number of ways) **[Medium]**
316. Perfect Squares **[Medium]**
317. Integer Break **[Medium]**
318. Decode Ways **[Medium]**
319. Decode Ways II **[Hard]**
320. Ugly Number III **[Medium]**
321. N-th Tribonacci Number **[Easy]**

9.2 2-D DP (Grid / Two Sequences)

Pattern: 2-D DP — state is two indices (i, j); fill a table row by row.

322. Unique Paths **[Medium]**
323. Unique Paths II (with obstacles) **[Medium]**
324. Minimum Path Sum in Grid **[Medium]**
325. Dungeon Game (reverse DP) **[Hard]**
326. Maximal Square **[Medium]**
327. Count Square Submatrices with All Ones **[Medium]**
328. Longest Common Subsequence **[Medium]**
329. Shortest Common Supersequence **[Hard]**
330. Edit Distance **[Hard]**
331. Distinct Subsequences **[Hard]**
332. Interleaving String **[Medium]**
333. Regular Expression Matching (DP) **[Hard]**
334. Wildcard Matching (DP) **[Hard]**
335. Scramble String **[Hard]**
336. Burst Balloons (interval DP) **[Hard]**

9.3 Knapsack Variants

Pattern: 0/1 Knapsack / Unbounded Knapsack — classic DP on items with weight/value.

337. 0/1 Knapsack Problem **[Medium]**
338. Unbounded Knapsack **[Medium]**
339. Partition Equal Subset Sum **[Medium]**
340. Target Sum (assign +/- to reach target) **[Medium]**
341. Last Stone Weight II (knapsack) **[Medium]**
342. Ones and Zeroes (2D knapsack) **[Medium]**
343. Profitable Schemes **[Hard]**
344. Number of Dice Rolls with Target Sum **[Medium]**
345. Combination Sum IV **[Medium]**
346. Count Ways to Build Good Strings **[Medium]**

9.4 Interval / Range DP

Pattern: Interval DP — solve subproblems over all intervals [i, j], merge results.

347. Matrix Chain Multiplication **[Hard]**
348. Minimum Cost Tree from Leaf Values **[Medium]**
349. Strange Printer **[Hard]**
350. Remove Boxes **[Hard]**
351. Zuma Game **[Hard]**
352. Minimum Cost to Merge Stones **[Hard]**
353. Optimal Strategy for a Game **[Medium]**
354. Palindrome Partitioning II (min cuts) **[Hard]**
355. Palindrome Partitioning IV (3 parts) **[Hard]**
356. Maximum Coins You Can Get **[Medium]**

9.5 State-Machine DP (Stocks & Transactions)

Pattern: State Machine DP — model allowed states (holding/not holding) and transitions.

357. Best Time to Buy and Sell Stock III (at most 2 transactions) **[Hard]**
358. Best Time to Buy and Sell Stock IV (at most k transactions) **[Hard]**
359. Best Time to Buy and Sell Stock with Cooldown **[Medium]**
360. Best Time to Buy and Sell Stock with Transaction Fee **[Medium]**
361. Paint House **[Medium]**
362. Paint House II (k colors) **[Hard]**
363. Paint Fence **[Medium]**
364. Student Attendance Record II **[Hard]**
365. Coin Path (min cost with k steps) **[Hard]**
366. Number of Ways to Stay in the Same Place After Some Steps **[Hard]**

9.6 Bitmask DP

Pattern: Bitmask DP — represent subsets as bitmasks; ideal for small n (≤20).

367. Traveling Salesman Problem (bitmask DP) **[Hard]**
368. Partition to K Equal Sum Subsets **[Medium]**
369. Minimum XOR Sum of Two Arrays (assignment) **[Hard]**
370. Maximum Students Taking Exam **[Hard]**
371. Stickers to Spell Word **[Hard]**
372. Shortest Path Visiting All Nodes (BFS + bitmask) **[Hard]**
373. Number of Ways to Wear Different Hats to Each Other **[Hard]**
374. Count Ways to Distribute Candies **[Hard]**
375. Maximize Score After N Operations **[Hard]**
376. Find the Shortest Superstring (bitmask DP) **[Hard]**

9.7 DP on Trees & Graphs

377. Diameter of N-ary Tree **[Hard]**
378. Binary Tree Maximum Path Sum (revisited) **[Hard]**
379. Maximum Sum of 3 Non-Overlapping Subarrays **[Hard]**
380. Cherry Pickup **[Hard]**
381. Cherry Pickup II (two robots) **[Hard]**
382. Minimum Difficulty of a Job Schedule **[Hard]**
383. Build Array Where You Can Find the Maximum Exactly K Comparisons **[Hard]**
384. Number of Music Playlists **[Hard]**
385. Count Vowels Permutation **[Hard]**
386. Minimum Cost to Cut a Stick **[Hard]**

Section 10: Backtracking (Q391 – Q420)

10.1 Subsets & Combinations

Pattern: Backtracking — explore all choices recursively; prune branches that violate constraints.

387. Subsets **[Medium]**
388. Subsets II (with duplicates) **[Medium]**
389. Combinations **[Medium]**
390. Combination Sum **[Medium]**
391. Combination Sum II (each number once) **[Medium]**
392. Combination Sum III (exactly k numbers) **[Medium]**
393. Letter Combinations of a Phone Number **[Medium]**
394. Generate Parentheses **[Medium]**
395. Count Numbers with Unique Digits **[Medium]**
396. Beautiful Arrangement **[Medium]**

10.2 Permutations

Pattern: Permutation Backtracking — swap elements or use a visited array.

397. Permutations **[Medium]**
398. Permutations II (with duplicates) **[Medium]**
399. Next Permutation **[Medium]**
400. Permutation Sequence (kth permutation) **[Hard]**
401. Palindrome Partitioning **[Medium]**
402. Word Search **[Medium]**
403. N-Queens **[Hard]**
404. N-Queens II (count solutions) **[Hard]**
405. Sudoku Solver **[Hard]**
406. Remove Invalid Parentheses **[Hard]**

10.3 Advanced Backtracking

407. Expression Add Operators **[Hard]**
408. Restore IP Addresses **[Medium]**
409. Letter Tile Possibilities **[Medium]**
410. Splitting a String Into Descending Consecutive Values **[Medium]**
411. Number of Squareful Arrays **[Hard]**
412. Maximum Length of a Concatenated String with Unique Characters **[Medium]**
413. Tiling a Rectangle with the Fewest Squares **[Hard]**
414. Coloring a Border **[Medium]**
415. Factor Combinations **[Medium]**
416. Construct the Lexicographically Largest Valid Sequence **[Medium]**

Section 11: Greedy Algorithms (Q421 – Q445)

11.1 Interval & Scheduling Greedy

Pattern: Greedy Selection — always pick the locally optimal choice; prove exchange argument.

417. Activity Selection / Non-overlapping Intervals **[Medium]**
418. Minimum Number of Platforms Required **[Medium]**
419. Job Sequencing Problem **[Medium]**
420. Fractional Knapsack **[Easy]**
421. N meetings in one room **[Easy]**
422. Assign Cookies **[Easy]**
423. Lemonade Change **[Easy]**
424. Queue Reconstruction by Height **[Medium]**
425. Two City Scheduling **[Medium]**
426. Minimum Cost to Move Chips to Same Position **[Easy]**

11.2 String & Array Greedy

427. Remove K Digits (smallest number) **[Medium]**
428. Largest Number After Removing k Digits (largest) **[Medium]**
429. Maximum Units on a Truck **[Easy]**
430. Minimum Deletions to Make Character Frequencies Unique **[Medium]**
431. Minimum Number of Flips to Make Binary String Alternating **[Medium]**
432. Minimum Swaps to Balance Parentheses **[Medium]**
433. Maximum Score from Removing Substrings **[Medium]**
434. Wiggle Subsequence **[Medium]**
435. Dota2 Senate (greedy queue) **[Medium]**
436. Minimum Time to Make Rope Colorful **[Medium]**
437. Boats to Save People **[Medium]**
438. Advantage Shuffle **[Medium]**
439. Maximum Performance of a Team **[Hard]**
440. Minimize Maximum Pair Sum in Array **[Medium]**
441. Earliest Deadline First Scheduling **[Medium]**

Section 12: Hashing, Math & Bit Manipulation (Q446 – Q470)

12.1 HashMap / HashSet Patterns

Pattern: HashMap — O(1) lookups to avoid nested loops; track frequency, index, or pairs.

442. Two Sum (HashMap) **[Easy]**
443. Longest Consecutive Sequence **[Medium]**
444. 4Sum II (count tuples) **[Medium]**
445. Isomorphic Strings **[Easy]**
446. Word Pattern **[Easy]**
447. Bulls and Cows **[Medium]**
448. Brick Wall (maximum bricks not cut) **[Medium]**
449. Subarray Sum Equals K (revisited with HashMap) **[Medium]**
450. Contiguous Array (equal 0s and 1s) **[Medium]**
451. Maximum Size Subarray Sum Equals k **[Medium]**

12.2 Math & Number Theory

Pattern: Number Theory — prime sieves, GCD/LCM, modular arithmetic, combinatorics.

452. Count Primes (Sieve of Eratosthenes) **[Medium]**
453. Power of Two / Three / Four **[Easy]**
454. Excel Sheet Column Number **[Easy]**
455. Happy Number (Floyd's cycle) **[Easy]**
456. Ugly Number **[Easy]**
457. Reverse Integer **[Medium]**
458. Palindrome Number **[Easy]**
459. Factorial Trailing Zeroes **[Medium]**
460. Nth Digit **[Medium]**
461. Super Power (modular exponentiation) **[Medium]**

12.3 Bit Manipulation

Pattern: Bit Tricks — XOR, AND/OR masks, Brian Kernighan's algorithm for O(1) bit operations.

462. Single Number **[Easy]**
463. Single Number II (bit counting) **[Medium]**
464. Single Number III (two unique) **[Medium]**
465. Number of 1 Bits (Hamming Weight) **[Easy]**
466. Counting Bits (0 to n) **[Easy]**
467. Reverse Bits **[Easy]**
468. Missing Number **[Easy]**
469. Sum of Two Integers (without + operator) **[Medium]**
470. Maximum XOR of Two Numbers in Array **[Medium]**
471. UTF-8 Validation **[Medium]**

Section 13: System Design & Data Structure Design (Q471 – Q500)

13.1 Classic Design Problems

Pattern: OOP + Right Data Structures — combine hash maps, heaps, doubly linked lists for O(1) ops.

472. Design LRU Cache **[Medium]**
473. Design LFU Cache **[Hard]**
474. Design Twitter (top 10 tweets feed) **[Medium]**
475. Design a Hit Counter **[Medium]**
476. Design a Rate Limiter (token bucket) **[Medium]**
477. Design In-Memory File System **[Hard]**
478. Design Search Autocomplete System (Trie + heap) **[Hard]**
479. Design a Phone Directory **[Medium]**
480. Implement Trie with Count **[Medium]**
481. Design Underground System (check-in/check-out) **[Medium]**

13.2 Randomized & Probabilistic Structures

Pattern: Reservoir Sampling / Fisher-Yates — uniform random sampling in O(1) space.

482. Shuffle an Array **[Medium]**
483. Random Pick Index (reservoir sampling) **[Medium]**
484. Linked List Random Node **[Medium]**
485. Random Pick with Weight **[Medium]**
486. Insert Delete GetRandom O(1) **[Medium]**
487. Insert Delete GetRandom O(1) — Duplicates Allowed **[Hard]**

13.3 Stream / Online Algorithms

Pattern: Online Algorithms — process data one element at a time without looking back.

488. Moving Average from Data Stream **[Easy]**
489. Find Median from Data Stream (revisited) **[Hard]**
490. Kth Largest Element in a Stream (revisited) **[Easy]**
491. Design a Stack with Increment Operation **[Medium]**
492. Design a Stack with getMin() / getMax() **[Easy]**
493. Time-Based Key-Value Store **[Medium]**
494. Design an Ordered Stream **[Easy]**
495. Snapshot Array **[Medium]**
496. Design a Log Aggregation System **[Medium]**
497. First Unique Number in Stream **[Medium]**

13.4 Advanced Design Challenges

498. Word Filter (prefix + suffix search, Trie) **[Hard]**
499. Exam Room (seat assignment) **[Medium]**
500. My Calendar I / II / III **[Medium]**
501. Range Module (add/remove/query ranges) **[Hard]**
502. All O'one Data Structure (O(1) min/max string freq) **[Hard]**
503. Design a Food Rating System **[Medium]**
504. Stock Price Fluctuation (max/min with updates) **[Medium]**
505. Find Servers That Handled Most Requests **[Hard]**
506. Booking Concert Tickets in Groups **[Hard]**
507. Subrectangle Queries **[Medium]**

Appendix A: Topic Summary & Pattern Quick-Reference

Section	Topic	Q Range	Key Pattern(s)
1	Arrays	1 – 50	Two Pointers, Sliding Window, Prefix Sum, Kadane's
2	Strings	51 – 90	HashMap, Sliding Window, KMP, Monotonic Stack
3	Linked Lists	91 – 120	Fast & Slow Pointers, In-place Reversal, Merge
4	Stacks & Queues	121 – 145	Monotonic Stack, Monotonic Deque, BFS
5	Binary Search	146 – 175	Classic BS, Parametric Search, Bisect
6	Trees	176 – 231	DFS/BFS, Recursion, BST, Segment Tree, Trie
7	Heaps	232 – 255	Top-K (min-heap), Two-Heap Median
8	Graphs	256 – 310	BFS, DFS, Topo Sort, DSU, Dijkstra, MST
9	Dynamic Programming	311 – 390	1-D/2-D DP, Knapsack, Interval, Bitmask, State-Machine
10	Backtracking	391 – 420	Subset/Combo, Permutation, Constraint Pruning
11	Greedy	421 – 445	Interval Greedy, Exchange Argument
12	Hashing & Math	446 – 470	HashMap, Number Theory, Bit Manipulation
13	Design	471 – 500	OOP + HashMap/Heap/Trie, Reservoir Sampling

Appendix B: Recommended 12-Week Study Plan

Week 1–2: Arrays (Q1–Q50) + Strings (Q51–Q90) — master sliding window and two-pointer patterns.
Week 3: Linked Lists (Q91–Q120) + Stacks & Queues (Q121–Q145) — pointer manipulation and monotonic structures.
Week 4: Binary Search (Q146–Q175) — classic and parametric search; practice writing bug-free lo/hi/mid logic.
Week 5–6: Trees (Q176–Q231) + Heaps (Q232–Q255) — DFS/BFS recursion, BST operations, Trie, priority queues.
Week 7–8: Graphs (Q256–Q310) — BFS/DFS, topological sort, Union-Find, Dijkstra, MST.
Week 9–10: Dynamic Programming (Q311–Q390) — start with 1-D, progress to 2-D, knapsack, interval, bitmask.
Week 11: Backtracking (Q391–Q420) + Greedy (Q421–Q445) — pruning strategies, exchange argument proofs.
Week 12: Hashing & Math (Q446–Q470) + Design (Q471–Q500) — mock interviews; time yourself strictly.

Closing Remarks

Consistent, pattern-focused practice is the most reliable path to success in FAANG-level technical interviews. Rather than memorising solutions, internalise the underlying pattern for each problem category. When you encounter an unfamiliar problem, identify which pattern it resembles and apply the corresponding template. Complement this list with timed mock interviews, peer code reviews, and systematic complexity analysis (time and space) for every solution you write.

Best of luck with your preparation. You have everything you need — now execute.